Spark Structured Streaming

Full Lab Manual

Environment: docker-compose with zookeeper, kafka, and jupyter

Lab 0 — Setup (do this once)

Start the environment:

```
docker compose up -d
```

Wait about 30 seconds, then open the notebook at http://localhost:8888/?token=training123

Create the Kafka topic used by every Kafka lab below:

```
docker exec -it kafka kafka-topics --create \
  --topic orders \
  --bootstrap-server kafka:9092 \
  --partitions 1 --replication-factor 1
```

To send test messages into “orders” at any point, open a second terminal:

```
docker exec -it kafka kafka-console-producer --topic orders --bootstrap-server kafka:9092
```

This drops you into a prompt — type a line, hit Enter, it's published. Leave this terminal open; you'll reuse it for every Kafka lab.

Start every notebook cell with:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()
```

Important — about console sink output in Jupyter:

writeStream runs in a background thread, and Jupyter only captures stdout produced synchronously while a cell is executing. You have two ways to actually see the output:

```
# Option A: tail the container logs from a host terminal
docker logs -f jupyter
```

```
# Option B: poll the query from inside the notebook
import time
for _ in range(5):
    print(query.status)
    print(query.lastProgress)
```

```
time.sleep(3)
```

For Lab 1 specifically, the manual below uses the memory sink instead of console — it lets you SELECT results directly in a later cell, which is far less confusing than watching container logs. You can apply the same swap to any other lab if you'd rather not use two terminals.

Lab 1 — File Streaming

Goal: stream CSV files dropped into a folder.

Step 1 — start with a clean, empty watch folder. This matters: Spark's file source only reliably picks up files that appear after the stream starts watching, so always reset the folder before you start the query, especially if you're re-running this lab.

```
!rm -rf /tmp/input
!mkdir -p /tmp/input
```

Step 2 — define the stream and start it. We use the memory sink here (queryName gives you a table to SELECT from) instead of console, since console output doesn't show up inline in Jupyter — see the note in Lab 0 setup.

```
df = spark.readStream \
   .format("csv") \
   .option("header", "true") \
   .schema("name STRING, amount INT") \
   .load("/tmp/input")

query = df.writeStream \
   .format("memory") \
   .queryName("file_stream") \
   .start()
```

Run the cell — it starts in the background (non-blocking, since we didn't call .awaitTermination(), which is better in a notebook so you can keep running cells). Confirm it's actually running before moving on:

```
print(query.isActive)
```

Step 3 — now that the stream is already watching, write a file into the folder:

```
with open("/tmp/input/data1.csv", "w") as f:
   f.write("name,amount\nA,100\nB,200\n")
```

Step 4 — give Spark a few seconds to pick it up, then query the in-memory table:

```
import time
time.sleep(5)
spark.sql("SELECT * FROM file_stream").show()
```

Observe: the SELECT shows rows A,100 and B,200. Add more files the same way (use a new filename each time, e.g. data2.csv) and re-run the SELECT — each new file triggers a new micro-batch and the table grows.

If you'd rather match the PDF exactly and use the console sink, swap .format("memory").queryName("file_stream") for .format("console") — just remember you'll need docker logs -f jupyter or query.lastProgress to actually see the output, per the note in Lab 0.

When done: query.stop()

Note: the PDF's original code used .option("header", "true") without an explicit schema, which works too, but streaming sources generally want a fixed schema — added above for reliability.

Lab 2 — Kafka Streaming (Basic)

Goal: read raw Kafka messages and print them.

```
df = spark.readStream \
   .format("kafka") \
   .option("kafka.bootstrap.servers", "kafka:9092") \
   .option("subscribe", "orders") \
   .load()

df2 = df.selectExpr("CAST(value AS STRING)")

query = df2.writeStream \
   .format("console") \
   .start()
```

Switch to the producer terminal from Lab 0 and type:

```
order1,100
order2,200
```

**query2 = df2.writeStream **
**.format("memory") **
**.queryName("kafka_stream1") **
.start()

import time
time.sleep(5)
spark.sql("SELECT * FROM kafka_stream1").show()

Observe: each line appears in the notebook's console output almost instantly.

query.stop() when done.

Lab 3 — Triggers: Default vs. processingTime

Case 1 — Default trigger (as fast as possible)

```
query1 = df2.writeStream \
   .format("console") \
   .start()
```

Send a few lines in the producer terminal, one every couple of seconds.

Observe: each one prints almost immediately — no waiting.

query1.stop()

Case 2 — Fixed interval trigger

```
query2 = df2.writeStream \
   .format("console") \
   .trigger(processingTime="10 seconds") \
   .start()
```

Send a line right away in the producer terminal, then wait.

Observe: it doesn't print until the next 10-second boundary, even though the data arrived earlier. Send a couple more lines in quick succession — they'll all print together at the next tick.

query2.stop()

Lab 4 — Transformations + Stateful Aggregation

Goal: parse user,amount and count orders per user, with counts that keep growing.

```
from pyspark.sql.functions import split

parsed = df2.select(
   split("value", ",")[0].alias("user"),
   split("value", ",")[1].alias("amount")
)

counts = parsed.groupBy("user").count()

query = counts.writeStream \
   .outputMode("complete") \
   .format("console") \
   .start()
```

In the producer terminal, send:

```
user1,50
user1,30
user2,20
user1,10
```

Observe: the printed table shows the full running counts each batch (this is complete mode) — user1 climbs to 2, then 3; user2 shows up once it sends data. This is stateful — Spark remembers prior counts across batches.

query.stop()

Lab 5 — Checkpointing + Writing to a File Sink

Goal: persist the raw stream to Parquet with fault-tolerant recovery.

```
!mkdir -p /tmp/orders /tmp/chk
```

```
query = df.selectExpr("CAST(value AS STRING)") \
    .writeStream \
    .format("parquet") \
    .option("path", "/tmp/orders") \
    .option("checkpointLocation", "/tmp/chk") \
    .start()
```

Send a few more lines through the producer, then check the output:

```
!ls /tmp/orders
spark.read.parquet("/tmp/orders").show()
```

Observe: Parquet part-files appear, and reading them back shows your messages. If you stop the query and start it again with the same checkpoint path, it resumes cleanly instead of reprocessing or losing data — that's what the checkpoint buys you.

query.stop()

Lab 6 — Windowed Aggregation

Goal: count orders in rolling 10-second tumbling windows, using Kafka's built-in timestamp column (the time Kafka received the message).

```
from pyspark.sql.functions import window

windowed = df \
    .groupBy(window("timestamp", "10 seconds")) \
    .count()

query = windowed.writeStream \
    .outputMode("complete") \
    .format("console") \
    .start()
```

Send a handful of messages spaced a few seconds apart in the producer terminal.

Observe: the output groups counts into [window_start, window_end] buckets rather than one global total — messages sent in the same 10-second slot land in the same row.

query.stop()

Lab 7 — Watermarking (Late Data)

Goal: tolerate late-arriving data up to a bound, then drop anything later than that.

```
watermarked = df \
   .withWatermark("timestamp", "1 minute") \
   .groupBy(window("timestamp", "10 seconds")) \
   .count()

query = watermarked.writeStream \
   .outputMode("complete") \
   .format("console") \
   .start()
```

Send messages normally — since we're using Kafka's ingestion timestamp, “late” data isn't easy to simulate manually here, but the important behavioral point to verify is:

Observe: older window buckets stop appearing in the output roughly a minute after their window closes — Spark has dropped that state to bound memory usage, per the watermark.

query.stop()

Lab 8 — Writing Output Back to Kafka

Goal: re-publish processed data to a new topic.

First create the output topic:

```
docker exec -it kafka kafka-topics --create \
  --topic output \
  --bootstrap-server kafka:9092 \
  --partitions 1 --replication-factor 1
```

Then, back in the notebook — Kafka sink requires a key/value (or at least value) column of type string/binary:

```
query = df.selectExpr("CAST(value AS STRING) as value") \
   .writeStream \
   .format("kafka") \
   .option("kafka.bootstrap.servers", "kafka:9092") \
   .option("topic", "output") \
   .option("checkpointLocation", "/tmp/chk_output") \
   .start()
```

To see it arrive, open a third terminal as a consumer on “output”:

```
docker exec -it kafka kafka-console-consumer --topic output --bootstrap-server kafka:9092 --from-beginning
```

Send messages into orders as before — observe them reappear on output.

query.stop()

Lab 9 — Final Advanced Pipeline

Goal: the capstone — count orders per user in 10-second windows, tolerating 1 minute of lateness, updating only changed rows.

```
from pyspark.sql.functions import split, window

df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "orders") \
    .load()

df2 = df.selectExpr("CAST(value AS STRING) as val", "timestamp")

parsed = df2.select(
    split("val", ",")[0].alias("user"),
    split("val", ",")[1].alias("amount"),
    "timestamp"
)

result = parsed \
    .withWatermark("timestamp", "1 minute") \
    .groupBy(window("timestamp", "10 seconds"), "user") \
    .count()

query = result.writeStream \
    .outputMode("update") \
    .format("console") \
    .option("checkpointLocation", "/tmp/chk_final") \
    .start()
```

In the producer terminal, send lines like:

```
user1,10
user2,20
user1,5
```

Observe: unlike Lab 6's complete mode (which reprints the whole table every batch), update mode only prints rows that changed in that batch — a more realistic pattern for a live dashboard that doesn't want to redraw everything constantly.

query.stop()

Cleanup

Stop all running streaming queries in the notebook, then:

```
docker compose down -v
```

(-v also removes the named volumes if you want a totally clean slate next time.)